OAVS Computer Teacher (TGT) Exam

MCQ Practice Book

sizeof Operator in C

120 Questions | 6 Sections | 2 Mock Tests

Includes Explanations, Tricky & Compiler-Based Questions

Topics Covered:

Syntax • Memory Calculation • Variables & Data Types

Arrays • Structures & Unions • Pointers • Output Questions

Table of Contents

Section 1: Basics of sizeof Operator (10 Questions)

Section 2: sizeof with Primitive Data Types (15 Questions)

Section 3: sizeof with Arrays (15 Questions)

Section 4: sizeof with Structures and Unions (15 Questions)

Section 5: sizeof with Pointers and Dynamic Memory (10 Questions)

Section 6: Tricky & Output-Based Questions (20 Questions)

Mock Test 1: Timed Practice – 15 Questions (15 min)

Mock Test 2: Advanced Timed Practice – 15 Questions (15 min)

Appendix: Quick Reference Table (–)

Section 1: Basics of sizeof Operator

Q1. What is sizeof in C?

A) A function
B) A macro
C) A unary operator
D) A keyword

> ■ **Correct Answer: C) A unary operator**
>
> **Explanation:** sizeof is a compile-time unary operator that returns the size (in bytes) of its operand.

Q2. What does sizeof return?

A) int
B) size_t
C) unsigned int
D) long

> ■ **Correct Answer: B) size_t**
>
> **Explanation:** sizeof returns a value of type size_t, which is an unsigned integer type defined in .

Q3. sizeof is evaluated at:

A) Run time
B) Link time
C) Compile time
D) Load time

> ■ **Correct Answer: C) Compile time**
>
> **Explanation:** sizeof is a compile-time operator; the compiler replaces it with a constant value before program execution.

Q4. Which header defines size_t?

A)
B)
C)
D)

> ■ **Correct Answer: C)**
>
> **Explanation:** size_t is defined in , though it is also indirectly available via several other standard headers.

Q5. What is the correct syntax of sizeof with a data type?

A) sizeof int

B) sizeof(int)

C) sizeof[int]

D) size(int)

> ■ **Correct Answer: B) sizeof(int)**
>
> **Explanation:** When used with a type name, parentheses are mandatory: sizeof(type).

Q6. Can sizeof be applied to a variable without parentheses?

A) No, never

B) Yes, sizeof x is valid

C) Only with pointers

D) Only in C99

> ■ **Correct Answer: B) Yes, sizeof x is valid**
>
> **Explanation:** When the operand is an expression (variable), parentheses are optional: sizeof x == sizeof(x).

Q7. What does sizeof('A') return on most compilers (C)?

A) 1

B) 2

C) 4

D) 8

> ■ **Correct Answer: C) 4**
>
> **Explanation:** In C, character literals are of type int (not char), so sizeof('A') == sizeof(int) == 4 on most 32-bit/64-bit systems.

Q8. sizeof applied to an expression does NOT evaluate the expression. This means:

A) It causes undefined behaviour

B) Side effects inside sizeof are ignored

C) It returns 0

D) The compiler throws an error

> ■ **Correct Answer: B) Side effects inside sizeof are ignored**
>
> **Explanation:** sizeof only needs the type of the expression; the expression is never actually executed at runtime, so i++ inside sizeof does not increment i.

Q9. Which of the following is NOT a valid use of sizeof?

A) sizeof(int)

B) sizeof(3+4.0)

C) sizeof(void)

D) sizeof(char *)

■ **Correct Answer: C) sizeof(void)**

Explanation: sizeof(void) is undefined/illegal in standard C; void has no size.

Q10. The result of sizeof is of type:

A) int

B) long

C) size_t

D) ptrdiff_t

■ **Correct Answer: C) size_t**

Explanation: The C standard mandates that sizeof returns size_t, an unsigned type large enough to hold the size of any object.

Section 2: sizeof with Primitive Data Types

Q11. What is sizeof(char) in C?

A) 1
B) 2
C) 4
D) Platform-dependent, but at least 1

■ Correct Answer: D) Platform-dependent, but at least 1

Explanation: The C standard guarantees sizeof(char) == 1 (by definition), but 1 byte can be more than 8 bits on exotic platforms.

Q12. sizeof(int) on a typical 32-bit system is:

A) 2
B) 4
C) 8
D) 1

■ Correct Answer: B) 4

Explanation: On most 32-bit and 64-bit systems, int is 4 bytes. The exact value depends on the platform/compiler.

Q13. sizeof(long double) is typically:

A) 8
B) 10
C) 12
D) 16

■ Correct Answer: C) 12

Explanation: On x86 Linux with GCC, sizeof(long double) is 12 bytes (80-bit extended with padding). On Windows MSVC it is 8.

Q14. Which data type always has sizeof equal to 1?

A) short
B) bool
C) char
D) unsigned char

■ Correct Answer: C) char

Explanation: By definition in the C standard, sizeof(char) is always 1.

Q15. What is sizeof(float)?

A) 2
B) 4
C) 8
D) 16

> ■ **Correct Answer: B) 4**
>
> **Explanation:** float is typically a 32-bit (4-byte) IEEE 754 single-precision value.

Q16. What is sizeof(double)?

A) 4
B) 6
C) 8
D) 16

> ■ **Correct Answer: C) 8**
>
> **Explanation:** double is an IEEE 754 64-bit (8-byte) double-precision floating-point type.

Q17. sizeof(short) on most platforms is:

A) 1
B) 2
C) 4
D) 8

> ■ **Correct Answer: B) 2**
>
> **Explanation:** short is guaranteed at least 16 bits (2 bytes) and is almost universally 2 bytes.

Q18. sizeof(long long) is:

A) 4
B) 8
C) 16
D) Compiler-defined

> ■ **Correct Answer: B) 8**
>
> **Explanation:** C99 introduced long long, which is guaranteed to be at least 64 bits (8 bytes).

Q19. sizeof(unsigned int) compared to sizeof(int) is:

A) Always smaller
B) Always larger
C) Always equal
D) Depends on sign

> ■ **Correct Answer: C) Always equal**

Explanation: The signed/unsigned qualifier does not change the storage size; both int and unsigned int occupy the same number of bytes.

Q20. If sizeof(int) == 4, what is sizeof(int *) on a 64-bit system?

A) 4

B) 8

C) Same as sizeof(int)

D) 2

■ Correct Answer: B) 8

Explanation: On a 64-bit system, any pointer type (including int *) is 8 bytes, regardless of the pointed-to type's size.

Q21. sizeof(bool) in C (with) is typically:

A) 1

B) 4

C) 8

D) Not defined in C

■ Correct Answer: A) 1

Explanation: In C99 and later, _Bool (aliased as bool via stdbool.h) has sizeof 1.

Q22. Which relation always holds in C?

A) sizeof(int) >= sizeof(short)

B) sizeof(short) > sizeof(char)

C) sizeof(long) > sizeof(int)

D) sizeof(double) > sizeof(long double)

■ Correct Answer: A) sizeof(int) >= sizeof(short)

Explanation: The C standard guarantees sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long).

Q23. Output of: printf("%zu", sizeof(3.14));

A) 4

B) 8

C) 10

D) Compiler error

■ Correct Answer: B) 8

Explanation: 3.14 is a double literal in C, so sizeof(3.14) == sizeof(double) == 8 on most platforms.

Q24. Output of: printf("%zu", sizeof(3.14f));

A) 4

B) 8

C) 10

D) 2

> ■ **Correct Answer: A) 4**
>
> **Explanation:** The f suffix makes 3.14f a float literal, and sizeof(float) == 4.

Q25. Output of: printf("%zu", sizeof(0));

A) 1

B) 2

C) 4

D) 8

> ■ **Correct Answer: C) 4**
>
> **Explanation:** 0 is an int literal. sizeof(int) == 4 on typical systems.

Section 3: sizeof with Arrays

Q26. int arr[10]; What is sizeof(arr)?

A) 10

B) 40

C) sizeof(int)

D) 4

> ■ **Correct Answer: B) 40**
>
> **Explanation:** sizeof(arr) gives the total size: 10 elements x sizeof(int) = 10 x 4 = 40 bytes.

Q27. How do you find the number of elements in an array using sizeof?

A) sizeof(arr)/sizeof(arr[0])

B) sizeof(arr)/sizeof(int)

C) Both A and B

D) sizeof(*arr)

> ■ **Correct Answer: C) Both A and B**
>
> **Explanation:** Both sizeof(arr)/sizeof(arr[0]) and sizeof(arr)/sizeof(int) correctly calculate the number of elements for an int array.

Q28. char str[] = "Hello"; What is sizeof(str)?

A) 5

B) 6

C) 4

D) 8

> ■ **Correct Answer: B) 6**
>
> **Explanation:** "Hello" is 5 characters plus the null terminator '\0', so sizeof(str) == 6.

Q29. void foo(int arr[]) { printf("%zu", sizeof(arr)); } What is printed?

A) Total array size

B) sizeof(int *)

C) Number of elements

D) 0

> ■ **Correct Answer: B) sizeof(int *)**
>
> **Explanation:** When an array is passed to a function, it decays to a pointer. So sizeof(arr) inside the function gives sizeof(int *), not the full array size.

Q30. int a[3][4]; What is sizeof(a)?

A) 12

B) 48
C) 7
D) 16

■ **Correct Answer: B) 48**

Explanation: 3 rows x 4 columns = 12 elements of int; 12 x 4 bytes = 48 bytes.

Q31. int a[3][4]; What is sizeof(a[0])?

A) 4
B) 12
C) 16
D) 48

■ **Correct Answer: C) 16**

Explanation: a[0] is a row of 4 ints: 4 x sizeof(int) = 4 x 4 = 16 bytes.

Q32. int a[3][4]; What is sizeof(a[0][0])?

A) 4
B) 12
C) 16
D) 48

■ **Correct Answer: A) 4**

Explanation: a[0][0] is a single int element, so sizeof(a[0][0]) == sizeof(int) == 4.

Q33. What is sizeof("ABC")?

A) 3
B) 4
C) 6
D) 8

■ **Correct Answer: B) 4**

Explanation: "ABC" is a string literal: 3 characters + null terminator = 4 bytes.

Q34. int *p = arr; sizeof(p) vs sizeof(arr) — which is larger?

A) sizeof(p) is larger
B) sizeof(arr) is larger (arr has multiple elements)
C) Always equal
D) Depends on int size

■ **Correct Answer: B) sizeof(arr) is larger (arr has multiple elements)**

Explanation: sizeof(arr) gives the full array size; sizeof(p) gives pointer size (8 on 64-bit). For any non-trivially-small array sizeof(arr) > sizeof(p).

Q35. double d[5]; How many elements can be computed as sizeof(d)/sizeof(*d)?

A) 5

B) 40

C) 8

D) 1

■ Correct Answer: A) 5

Explanation: sizeof(d) = 5*8 = 40; sizeof(*d) = sizeof(double) = 8; 40/8 = 5 elements.

Q36. int arr[] = {1,2,3,4,5}; int n = sizeof(arr)/sizeof(arr[0]); n = ?

A) 4

B) 5

C) 20

D) 1

■ Correct Answer: B) 5

Explanation: sizeof(arr) = 20 (5 ints x 4), sizeof(arr[0]) = 4, so n = 20/4 = 5.

Q37. char c[100]; What is sizeof(c)?

A) 1

B) 4

C) 100

D) 101

■ Correct Answer: C) 100

Explanation: c is a fixed-size char array of 100 elements; sizeof(c) == 100.

Q38. sizeof on a VLA (Variable Length Array) is evaluated at:

A) Compile time

B) Runtime

C) Link time

D) Preprocessor stage

■ Correct Answer: B) Runtime

Explanation: For VLAs (C99), sizeof is evaluated at runtime because the array size is not known at compile time.

Q39. int n=5; int vla[n]; sizeof(vla) is evaluated at:

A) Compile time only

B) Runtime

C) Macro expansion time

D) Not allowed

> ■ **Correct Answer: B) Runtime**
>
> **Explanation:** vla is a Variable Length Array; sizeof(vla) is computed at runtime in this case.

Q40. Which of the following uses sizeof correctly to copy an array?

A) memcpy(dst, src, sizeof(src))

B) memcpy(dst, src, sizeof(*src))

C) Both if src is an array (not a pointer)

D) Neither

> ■ **Correct Answer: C) Both if src is an array (not a pointer)**
>
> **Explanation:** When src is a plain array (not decayed to a pointer), sizeof(src) gives total bytes; sizeof(*src) gives one element's size (so needs n * sizeof(*src) for full copy). Option A is correct for full copy if src is still an array.

Section 4: sizeof with Structures and Unions

Q41. struct { int a; char b; }; What is its sizeof likely to be?

A) 5
B) 8
C) 4
D) 6

■ Correct Answer: B) 8

Explanation: Due to structure padding, the compiler aligns char b to a 4-byte boundary after int a (4 bytes), adding 3 padding bytes: total 8.

Q42. What is structure padding?

A) Adding extra members
B) Extra bytes added by compiler for alignment
C) Zero-initialising members
D) Removing unused members

■ Correct Answer: B) Extra bytes added by compiler for alignment

Explanation: Compilers insert padding bytes between or after members to satisfy alignment requirements of the hardware.

Q43. How can you prevent structure padding in GCC?

A) #pragma pack(1)
B) __attribute__((packed))
C) Both A and B
D) sizeof cannot be controlled

■ Correct Answer: C) Both A and B

Explanation: Both #pragma pack(1) and __attribute__((packed)) instruct GCC to remove padding, though this may hurt performance.

Q44. struct { char a; int b; char c; }; sizeof is likely:

A) 6
B) 9
C) 12
D) 8

■ Correct Answer: C) 12

Explanation: char a (1) + 3 pad + int b (4) + char c (1) + 3 pad = 12 bytes on a typical system.

Q45. struct { int a; int b; int c; }; sizeof is:

A) 12

B) 8

C) 16

D) 4

> ■ **Correct Answer: A) 12**
>
> **Explanation:** Three consecutive int fields, each 4 bytes, no padding needed: 3 x 4 = 12 bytes.

Q46. union { int a; char b; double c; }; sizeof is:

A) 13

B) 8

C) 4

D) 16

> ■ **Correct Answer: B) 8**
>
> **Explanation:** A union's size equals the size of its largest member (double = 8), plus any required alignment padding.

Q47. sizeof(struct) vs sizeof(union) for the same members — which is usually larger?

A) struct

B) union

C) Always equal

D) Depends on member count

> ■ **Correct Answer: A) struct**
>
> **Explanation:** A struct allocates space for all members (plus padding); a union allocates space only for the largest member.

Q48. Bit-fields inside a struct affect sizeof because:

A) They do not affect sizeof at all

B) The compiler packs them but may add padding to meet alignment

C) sizeof counts bits, not bytes

D) Bit-fields are illegal in C

> ■ **Correct Answer: B) The compiler packs them but may add padding to meet alignment**
>
> **Explanation:** Bit-fields are packed but the containing unit and overall struct may be padded to natural alignment boundaries.

Q49. struct Empty {}; sizeof(struct Empty) in C is:

A) 0

B) 1

C) 4

D) Undefined behaviour

■ **Correct Answer: A) 0**

Explanation: In C, an empty struct has size 0 (though this is technically a GCC extension; C++ guarantees size >= 1).

Q50. Rearranging struct members can reduce sizeof. The general rule is:

A) Place larger types first

B) Place smaller types first

C) Order does not matter

D) Always use unions instead

■ **Correct Answer: A) Place larger types first**

Explanation: Placing larger (higher-alignment) members first reduces or eliminates inter-member padding gaps.

Q51. struct { short s; char c; }; sizeof is likely:

A) 2

B) 3

C) 4

D) 6

■ **Correct Answer: C) 4**

Explanation: short (2) + char (1) + 1 pad (to reach next 2-byte boundary for the struct) = 4 bytes.

Q52. struct { double d; char c; }; sizeof is likely:

A) 9

B) 12

C) 16

D) 8

■ **Correct Answer: C) 16**

Explanation: double d (8) + char c (1) + 7 pad (alignment of double is 8) = 16 bytes.

Q53. To find the offset of a member inside a struct, which macro is used?

A) sizeof

B) offsetof

C) alignof

D) typeof

■ **Correct Answer: B) offsetof**

Explanation: offsetof(type, member) from gives the byte offset of a member within a struct.

Q54. union { int i; float f; }; sizeof(union) == ?

A) 8

B) 4

C) 2

D) 1

> ■ **Correct Answer: B) 4**
>
> **Explanation:** Both int and float are 4 bytes, so the union size = max(4,4) = 4 bytes.

Q55. struct S { int x; struct S *next; }; sizeof(struct S) on a 64-bit system:

A) 8

B) 12

C) 16

D) 4

> ■ **Correct Answer: C) 16**
>
> **Explanation:** int x (4) + 4 pad (to align 8-byte pointer) + struct S *next (8) = 16 bytes.

Section 5: sizeof with Pointers and Dynamic Memory

Q56. On a 32-bit system, sizeof(int *) is:

A) 2
B) 4
C) 8
D) Same as sizeof(int)

> ■ **Correct Answer: B) 4**
>
> **Explanation:** On a 32-bit system, all pointers are 4 bytes because addresses are 32-bit wide.

Q57. On a 64-bit system, sizeof(char *) is:

A) 1
B) 4
C) 8
D) Same as sizeof(char)

> ■ **Correct Answer: C) 8**
>
> **Explanation:** On a 64-bit system, any pointer (including char *) is 8 bytes.

Q58. sizeof(int *) == sizeof(char *) — is this true?

A) Always true
B) Always false
C) Platform-dependent
D) True only in C99

> ■ **Correct Answer: A) Always true**
>
> **Explanation:** All data pointer types have the same size on a given platform; the C standard requires this for data pointers.

Q59. int *p = malloc(10 * sizeof(int)); sizeof(p) gives:

A) 40
B) 4 or 8 (pointer size)
C) 10
D) sizeof(int)

> ■ **Correct Answer: B) 4 or 8 (pointer size)**
>
> **Explanation:** p is a pointer; sizeof(p) gives the pointer's own size (4 or 8), NOT the size of the allocated memory.

Q60. To correctly allocate memory for n doubles, use:

A) malloc(n)

B) malloc(n * sizeof(double))

C) malloc(sizeof(double))

D) calloc(n, 1)

> ■ **Correct Answer: B) malloc(n * sizeof(double))**
>
> **Explanation:** malloc needs the total byte count; n * sizeof(double) computes the exact bytes needed for n doubles.

Q61. int **pp; sizeof(pp) on a 64-bit system is:

A) 4

B) 8

C) 16

D) Compiler error

> ■ **Correct Answer: B) 8**
>
> **Explanation:** pp is a pointer-to-pointer; it is still just a pointer, so sizeof(pp) == sizeof(void *) == 8 on 64-bit.

Q62. sizeof(NULL) in C is:

A) 0

B) sizeof(void *)

C) 1

D) Undefined

> ■ **Correct Answer: B) sizeof(void *)**
>
> **Explanation:** NULL is typically defined as (void *)0; sizeof(NULL) == sizeof(void *), which is 4 or 8.

Q63. void *vp; sizeof(vp) is:

A) 0

B) 1

C) sizeof(void)

D) Pointer size (4 or 8)

> ■ **Correct Answer: D) Pointer size (4 or 8)**
>
> **Explanation:** vp is a void pointer; sizeof(vp) gives the pointer's size, not the size of what it points to.

Q64. sizeof(int(*)[10]) on a 64-bit system is:

A) 40

B) 8

C) 4

D) 10

■ **Correct Answer: B) 8**

Explanation: int(*)[10] is a pointer to an array of 10 ints; it is still a single pointer, so sizeof == 8 on 64-bit.

Q65. int arr[5]; int *p = arr; sizeof(arr)/sizeof(*arr) vs sizeof(p)/sizeof(*p) — which gives the correct element count?

A) Both give 5

B) Only sizeof(arr)/sizeof(*arr)

C) Only sizeof(p)/sizeof(*p)

D) Neither

■ **Correct Answer: B) Only sizeof(arr)/sizeof(*arr)**

Explanation: sizeof(arr) gives 20 (5*4); sizeof(p) gives 8 (pointer size). p/(*p) gives 8/4 = 2, which is wrong. Only the array version is correct.

Section 6: Tricky & Output-Based Questions

Q66. int i = 5; int x = sizeof(i++); After this, i = ?

A) 5
B) 6
C) 4
D) Undefined

> ■ **Correct Answer: A) 5**
>
> **Explanation:** sizeof does not evaluate its operand at runtime; i++ is never executed, so i remains 5.

Q67. sizeof(1==1) in C returns:

A) 1
B) 4
C) 2
D) 0

> ■ **Correct Answer: B) 4**
>
> **Explanation:** (1==1) is an int expression in C (C uses int for boolean results, not bool), so sizeof == sizeof(int) == 4.

Q68. sizeof(sizeof(int)) returns:

A) 4
B) 8
C) sizeof(size_t)
D) Compiler error

> ■ **Correct Answer: C) sizeof(size_t)**
>
> **Explanation:** sizeof(int) returns a value of type size_t; sizeof applied to that returns sizeof(size_t), typically 8 on 64-bit.

Q69. enum Color { RED, GREEN, BLUE }; sizeof(Color) is typically:

A) 1
B) 4
C) 12
D) 3

> ■ **Correct Answer: B) 4**
>
> **Explanation:** Enum types are typically represented as int in C, so sizeof == sizeof(int) == 4.

Q70. What does sizeof a function return in C?

A) Size of return type

B) Undefined / error

C) 1

D) 4

> ■ **Correct Answer: B) Undefined / error**
>
> **Explanation:** Applying sizeof to a function is undefined behaviour in C; functions are not objects.

Q71. printf("%d", sizeof(long) > sizeof(int)); Output (64-bit Linux)?

A) 0

B) 1

C) -1

D) Compiler error

> ■ **Correct Answer: B) 1**
>
> **Explanation:** On 64-bit Linux, sizeof(long)==8 and sizeof(int)==4, so 8>4 is 1 (true).

Q72. int a=2, b=3; sizeof(a+b) returns:

A) sizeof(int)

B) sizeof(a)+sizeof(b)

C) 2

D) 6

> ■ **Correct Answer: A) sizeof(int)**
>
> **Explanation:** a+b is an int expression; sizeof computes the type, not the value. Result == sizeof(int) == 4.

Q73. What is sizeof(struct) if struct has only one char member?

A) 0

B) 1

C) 4

D) Platform-dependent

> ■ **Correct Answer: B) 1**
>
> **Explanation:** A struct with a single char member has no padding needed and occupies exactly 1 byte.

Q74. char *s = "Hello"; sizeof(s) vs sizeof("Hello") — which is larger?

A) sizeof(s)

B) sizeof("Hello")

C) Always equal

D) Depends on platform

■ **Correct Answer: B) sizeof("Hello")**

Explanation: sizeof("Hello") == 6 (5 chars + null); sizeof(s) == pointer size (8 on 64-bit or 4 on 32-bit). On 32-bit sizeof(s)==4<6; on 64-bit both may differ.

Q75. sizeof(int[0]) is:

A) 0

B) 4

C) 1

D) Compiler error

■ **Correct Answer: A) 0**

Explanation: An array of zero elements has size 0 (GCC extension; strictly speaking it is a C99 flexible array member scenario).

Q76. int x; sizeof(x=5) returns — and x after this line?

A) 4, x=5

B) 4, x unchanged

C) 1, x=5

D) Undefined

■ **Correct Answer: B) 4, x unchanged**

Explanation: sizeof does not evaluate x=5; the assignment never happens. sizeof returns sizeof(int) == 4.

Q77. Which format specifier should be used with sizeof in printf?

A) %d

B) %i

C) %zu

D) %lu

■ **Correct Answer: C) %zu**

Explanation: %zu is the correct format specifier for size_t in C99 and later.

Q78. int arr[10]; passed to a function. Inside the function, sizeof(arr)/sizeof(arr[0]) gives:

A) 10

B) 2 (on 64-bit)

C) 4 (on 32-bit)

D) Both B and C depending on platform

■ **Correct Answer: D) Both B and C depending on platform**

Explanation: Inside the function, arr decays to int *; sizeof(int*)/sizeof(int) = 8/4 = 2 on 64-bit, or 4/4 = 1 on 32-bit — not 10.

Q79. typedef int MyInt; sizeof(MyInt) is:

A) sizeof of pointer

B) sizeof(int)

C) sizeof(char)

D) Compiler error

> ■ **Correct Answer: B) sizeof(int)**
>
> **Explanation:** typedef creates an alias; sizeof(MyInt) == sizeof(int) == 4.

Q80. struct { int : 0; }; sizeof this struct is:

A) 0

B) 4

C) 1

D) Undefined

> ■ **Correct Answer: A) 0**
>
> **Explanation:** A zero-width bit-field forces alignment but adds no storage; an otherwise-empty struct has size 0 in GCC C mode.

Q81. #define SZ sizeof; SZ(int) compiles?

A) Yes, SZ(int) == 4

B) No, sizeof is not a function

C) Depends on compiler

D) Macro expansion fails

> ■ **Correct Answer: A) Yes, SZ(int) == 4**
>
> **Explanation:** sizeof is an operator but syntactically looks like a function call; the macro substitution SZ(int) -> sizeof(int) is valid.

Q82. sizeof(main) in C is:

A) 4

B) 8

C) Undefined behaviour

D) 1

> ■ **Correct Answer: C) Undefined behaviour**
>
> **Explanation:** sizeof applied to a function is not defined in the C standard.

Q83. int a = sizeof; — is this valid?

A) Yes

B) No

C) Depends on compiler

D) Only in C11

> ■ **Correct Answer: B) No**
>
> **Explanation:** sizeof is a keyword/operator, not a value; it cannot be stored in a variable.

Q84. In C, sizeof(float) == sizeof(int). This is:

A) Always true

B) Always false

C) True on most common platforms

D) Never true

> ■ **Correct Answer: C) True on most common platforms**
>
> **Explanation:** On most 32/64-bit platforms, both float and int are 4 bytes, but the C standard does not guarantee this.

Q85. What is the output? int a[3]={1,2,3}; printf("%zu",sizeof(a)/sizeof(0.0+a[0]));

A) 3

B) 1

C) 12

D) Compiler error

> ■ **Correct Answer: B) 1**
>
> **Explanation:** 0.0+a[0] promotes a[0] to double; sizeof(double)==8. sizeof(a)==12. 12/8 = 1 (integer division), not 3!

Mock Test 1 – 15 Questions (Time: 15 Minutes)

This is a timed practice test. Attempt all questions, then verify with the answer key.

MT1-Q1. sizeof is a ___ in C.

A) Function
B) Macro
C) Unary compile-time operator
D) Library routine

MT1-Q2. sizeof(char) is always:

A) 0
B) 1
C) 2
D) 4

MT1-Q3. int a[7]; sizeof(a) on a 32-bit system:

A) 7
B) 14
C) 28
D) 56

MT1-Q4. char *p; sizeof(p) on 64-bit:

A) 1
B) 4
C) 8
D) 2

MT1-Q5. sizeof(3.0) in C equals:

A) sizeof(float)
B) sizeof(double)
C) sizeof(long double)
D) 4

MT1-Q6. struct{ char a; int b; }; typical sizeof:

A) 5
B) 8
C) 4
D) 6

MT1-Q7. int i=0; sizeof(i++); after this line, i is:

A) 0
B) 1

C) 2

D) Undefined

MT1-Q8. Correct printf format for sizeof result:

A) %d

B) %u

C) %zu

D) %ld

MT1-Q9. sizeof(union) equals size of:

A) Smallest member

B) Largest member

C) Sum of all members

D) Average member size

MT1-Q10. double d[4]; sizeof(d)/sizeof(d[0]) = ?

A) 4

B) 8

C) 32

D) 1

MT1-Q11. sizeof("Hi") = ?

A) 2

B) 3

C) 4

D) 8

MT1-Q12. On a 64-bit system sizeof(int*) == sizeof(char*)?

A) True

B) False

C) Depends

D) Compiler error

MT1-Q13. To find array length use:

A) sizeof(arr)

B) sizeof(arr)/sizeof(arr[0])

C) length(arr)

D) arr.length

MT1-Q14. sizeof a VLA is computed at:

A) Compile time

B) Runtime

C) Link time

D) Preprocessor time

MT1-Q15. int x; sizeof(x=10) returns (and x changes?)

A) 4, x=10

B) 4, x unchanged

C) 10, x=10

D) 0, undefined

Answer Key

1. C **2.** B **3.** C **4.** C **5.** B **6.** B **7.** A **8.** C **9.** B **10.** A **11.** B **12.** A **13.** B **14.** B **15.** B

Mock Test 2 – 15 Questions (Time: 15 Minutes)

Advanced level. Focus on tricky compiler-based questions.

MT2-Q1. sizeof(sizeof(int)) equals:

A) 4
B) sizeof(size_t)
C) 8 always
D) sizeof(int)

MT2-Q2. int a[3][3]; sizeof(a[1]) = ?

A) 4
B) 9
C) 12
D) 36

MT2-Q3. struct{int a;char b;double c;}; typical sizeof on 64-bit:

A) 13
B) 16
C) 24
D) 20

MT2-Q4. sizeof(1==1) in C:

A) 1
B) 4
C) 2
D) 0

MT2-Q5. typedef double D; sizeof(D) =

A) sizeof(double *)
B) sizeof(double)
C) 4
D) 2

MT2-Q6. In a function receiving int arr[], sizeof(arr) gives:

A) Full array size
B) sizeof(int *)
C) Number of elements
D) 0

MT2-Q7. What does #pragma pack(1) do to sizeof?

A) Increases it
B) Removes padding, may decrease it

C) Has no effect

D) Causes error

MT2-Q8. enum { A,B,C }; sizeof(enum) typically:

A) 1

B) 3

C) 4

D) 8

MT2-Q9. sizeof(long double) on Windows MSVC:

A) 8

B) 10

C) 12

D) 16

MT2-Q10. int x=2,y=3; sizeof(x,y) = ?

A) sizeof(int)

B) sizeof(int)+sizeof(int)

C) sizeof(y) because comma evaluates to y

D) Error

MT2-Q11. char c='A'; sizeof(c) in C:

A) 4

B) 1

C) 2

D) Same as sizeof('A')

MT2-Q12. malloc(n * sizeof(*ptr)) is preferred because:

A) It is faster

B) It adapts automatically if ptr type changes

C) sizeof(*ptr) is always 1

D) It avoids null pointer

MT2-Q13. sizeof(int)-sizeof(long) could be negative — but stored in size_t, it wraps to:

A) 0

B) A very large number

C) -4

D) Compiler error

MT2-Q14. int arr[5]; What prints? printf("%zu",sizeof arr);

A) 5

B) 20

C) 4

D) 40

MT2-Q15. Which correctly counts struct array elements?

A) sizeof(s)/sizeof(struct S)

B) sizeof(s)/sizeof(s[0])

C) Both A and B

D) Neither

Answer Key

1. B **2.** C **3.** B **4.** B **5.** B **6.** B **7.** B **8.** C **9.** A **10.** C **11.** B **12.** B **13.** B **14.** B **15.** C

Appendix: Quick Reference — sizeof Cheat Sheet

Expression	32-bit System	64-bit System	Notes
sizeof(char)	1	1	Always 1
sizeof(short)	2	2	
sizeof(int)	4	4	
sizeof(long)	4	8	Differs on LP64 (Linux)
sizeof(long long)	8	8	C99 minimum 8
sizeof(float)	4	4	IEEE 754 single
sizeof(double)	8	8	IEEE 754 double
sizeof(long double)	12	16	GCC x86: 12, x86-64: 16
sizeof(pointer)	4	8	All data pointer types
sizeof(bool)	1	1	C99 _Bool
sizeof(void *)	4	8	Function ptr may differ

Key Rules to Remember:

1. sizeof is a compile-time operator (except for VLAs in C99).
2. sizeof always returns size_t (use %zu in printf).
3. sizeof does NOT evaluate its operand — side effects are ignored.
4. sizeof(char) is always 1 by the C standard.
5. Arrays decay to pointers inside functions — use sizeof in the calling scope.
6. Structures may have padding; use offsetof() to find member offsets.
7. Unions are sized by their largest member.
8. To find array length: sizeof(arr) / sizeof(arr[0]).
9. Character literals in C have type int, so sizeof('A') == sizeof(int).
10. Use sizeof(*ptr) in malloc for type-safe allocation: malloc(n * sizeof(*ptr)).